

AI Usage Summary

This document describes how AI assistance was used during development.

Prompt 1: Backend Architecture Design

Prompt

“Design a Spring Boot transaction management backend that stores transactions in JSON files instead of a database and supports concurrent access.”

Result

The AI suggested:

- Using ConcurrentHashMap for in-memory storage.
 - Maintaining a secondary index by account ID.
 - Using BlockingQueue for asynchronous persistence.
 - Writing updates periodically using a scheduled task.
 - Keeping runtime data outside the application package.
-

Prompt 2: JSON Persistence Strategy

Prompt

“How should a Spring Boot application handle writable JSON files when resources inside the project are read-only?”

Result

The AI recommended:

- Keeping an internal JSON file as initial data.
 - Creating an external writable JSON file at runtime.
 - Loading from the external file after initialization.
 - Using atomic file replacement when saving.
-

Prompt 3: Concurrent Data Management

Prompt

“How can transactions be stored efficiently in memory and retrieved by account?”

Result

The AI recommended:

Primary storage:

`ConcurrentHashMap<TransactionId, Transaction>`

Secondary index:

`ConcurrentHashMap<AccountId, Set<TransactionId>>`

This avoids scanning all transactions when retrieving by account.

Prompt 4: Persistence Buffer Design

Prompt

“Should transaction changes be stored using a dirty flag or a queue before persistence?”

Result

The AI recommended:

Using:

`BlockingQueue<Transaction>`

because:

- It supports future asynchronous processing.
 - It allows batching.
 - It can later be replaced by Kafka or another messaging system.
-

Prompt 5: Logging Design

Prompt

“How should logging be implemented for transaction operations?”

Result

The AI suggested separating:

Technical logging

Using SLF4J:

`application.log`

For:

- application startup
- repository operations
- persistence events

Business audit logging

Using a dedicated logger:

`transactions.log`

For:

- transaction creation
- transaction reversal
- transaction updates

Error logging

Using:

`errors.log`

For unexpected failures.

Prompt 6: Exception Handling

Prompt

“How should `GlobalExceptionHandler` logging be implemented?”

Result

The AI suggested:

- Business exceptions → WARN logs.
- Unexpected exceptions → ERROR logs.

- Avoid duplicate logging.
 - Keep exception logging centralized.
-

Prompt 7: Frontend Development

Prompt

“Create a responsive React frontend for transaction listing, creation, and reversal.”

Result

The AI proposed:

- Single dashboard approach.
 - Transaction table with filtering.
 - Transaction creation form.
 - Reverse transaction action.
 - Responsive UI design.
-

Summary

AI was used as a development assistant for:

- Architecture decisions
- Code review
- Performance considerations
- Logging strategy
- Persistence design
- Exception handling
- Frontend structure

All generated suggestions were reviewed, adapted, and integrated into the final implementation.